

Algorithme en Java

Consolider les fondamentaux du langage par la pratique

En bref

Une série d'exercices d'algorithme pour prendre en main la **syntaxe de Java et son typage**, en vous appuyant sur des raisonnements que vous connaissez déjà. **Jour 1** : résoudre les exercices. **Jour 2** : écrire les tests unitaires qui prouvent qu'ils fonctionnent (cf brief 2) .

1. Les exercices

Chaque exercice est une ou plusieurs **méthodes à écrire**. Concentrez-vous sur la logique et le typage, pas sur les bibliothèques. **Règle** : chaque méthode reçoit ses données en paramètre et renvoie un résultat. **Important** : Il y a plusieurs exercices, le but n'est pas de tous les faire mais d'aller le plus possible.

Exécution : un fichier par exercice, lancé en **ligne de commande**, en passant les variables en arguments. Le résultat s'affiche dans la console. Exemple : `java FizzBuzz.java 15` doit afficher `FizzBuzz`. (Astuce : entourer de guillemets les arguments contenant des caractères spéciaux, ex. `"Azerty123!xyz"`.)

Prérequis : Java accessible en ligne de commande

Avant de commencer, vérifiez dans le terminal que Java répond : tapez `java -version`. Si la version s'affiche, tout est prêt.

Si le terminal répond « java n'est pas reconnu », le JDK n'est pas dans le PATH. Ajoutez le dossier bin du JDK au PATH (rechercher « variables d'environnement », puis Path, Nouveau, coller le chemin du dossier bin), puis rouvrez le terminal.

Astuce : pour retrouver le chemin exact du JDK, ouvrez IntelliJ, menu File puis Project Structure puis SDKs.

Exercice 1 — Calculatrice (échauffement)

Écrire cinq méthodes avec `a` et `b` en entrée : `sum`, `subtraction`, `multiplication`, `division`, et `sumFromAtoB` (la somme de tous les entiers de `a` à `b` : pour 2 et 5, on additionne `2 + 3 + 4 + 5 = 14`). Le programme affiche **tous les résultats d'un coup**. **Notions** : déclarer une méthode, paramètres typés, valeur de retour, boucle (pour la somme).

Piège à observer : la **division** : que vaut `5 / 2` en Java avec des `int` ? Et avec des `double` ?

En ligne de commande : `java Calculator.java 2 5` affiche tous les résultats : Addition 7, Soustraction -3, Multiplication 10, Division 0.4, Somme de 2 à 5 : 14.

Exercice 2 — FizzBuzz

Méthode : `fizzBuzz(int n) : String`. Renvoyer « Fizz » si `n` est divisible par 3, « Buzz » par 5, « FizzBuzz » par 15, sinon le nombre lui-même. **Notions** : conditions, opérateur modulo (%).

En ligne de commande : `java FizzBuzz.java 15` → `FizzBuzz` | `java FizzBuzz.java 7` → `7`.

Exercice 3 — Manipulation de chaînes

Deux méthodes : `countOccurrences(String texte, char lettre) : int` et `isPalindrome(String mot) : boolean` (le mot se lit-il pareil dans les deux sens, ex. « radar »). **Notions** : parcours de chaîne, comparaison de caractères, retour booléen.

En ligne de commande : `java StringExplore.java mississippi s` → `4 false` | `java StringExplore.java radar r` → `2 true`.

Exercice 4 — Vérificateur de mot de passe

Méthode : `isPasswordValid(String pwd) : boolean`. Un mot de passe est valide s'il respecte **toutes** ces règles :

- au moins 12 caractères ;
- au moins une lettre ;
- au moins un chiffre ;
- au moins un caractère spécial (ni lettre ni chiffre) ;
- aucun espace.

Contrainte : sans expression régulière (regex). Parcourez la chaîne caractère par caractère et vérifiez les conditions à la main. **Notions** : conditions multiples, drapeaux booléens. Si plusieurs contraintes ne sont pas respectées un message doit s'afficher pour chaque.

En ligne de commande : `java MotDePasse.java "Azerty123!xyz" → true` | `java Azerty1! -> "Le mot de pass est trop court"` `"Le mot de passe doit contenir 12 caractère"` `false`

Exercice 5 — Dichotomie

Méthode : `numberOfTry(int cible, int min, int max) : int`. Le programme doit **deviner un nombre** avec le moins d'essai possible, et renvoyer le nombre d'essais. A chaque essai imprimé la tentative

Intervalle	Nombre d'essais maximum
1 à 100	7
1 à 200	8
1 à 500	9
1 à 1000	10

Notions : boucle, gestion des bornes (min, milieu, max), condition d'arrêt. **Astuce** : comparez le milieu à la cible, puis réduisez l'intervalle de moitié du bon côté.

En ligne de commande : `java Dichotomie.java 42 1 100 → Essai 1 : 50, le chiffre est plus petit"` `Essai 2 : 25, le chiffre est plus grand"..... 7` (nombre d'essais utilisés).

Exercice 6 — Chiffres romains (optionnel)

Méthode : `enRomain(int nombre) : String`. Convertir un entier en chiffres romains (4 → IV, 9 → IX, 14 → XIV, 40 → XL, 2024 → MMXXIV). **Notions** : décomposition, correspondances valeur/symbole, accumulation.

En ligne de commande : `java Romains.java 2024 → MMXXIV`.

Exercice 7 — Kata Potter simplifié (optionnel, défi avancé)

Une librairie vend chaque tome de Harry Potter à 8 €. Plus un panier contient de tomes **différents**, plus la remise est forte :

Tomes différents	Remise	Prix du lot
1	0 %	8 €
2	5 %	15,20 €
3	10 %	21,60 €
4	20 %	25,60 €
5	25 %	30 €

En ligne de commande : `java Potter.java 1 2 3 → 21.6`.

2. Modalités pédagogiques

Paramètre	Détail
Durée	1 jour pour les exercices, 1 jour pour les tests unitaires.
Organisation	Travail individuel, entraide encouragée.
Contrainte clé	Un fichier par exercice, exécuté en ligne de commande avec les variables en arguments (ex. java FizzBuzz.java 15). Chaque méthode reste pure, donc testable au Jour 2.
Ressources	Voir section 6.

Déroulé conseillé

1. Calculatrice (échauffement : méthodes, types, première boucle).
2. FizzBuzz.
3. Manipulation de chaînes (occurrences + palindrome).
4. Vérificateur de mot de passe.
5. Dichotomie
6. Chiffres romains, puis Kata Potter (facultatif)

3. Modalités d'évaluation

- **Justesse** : chaque méthode compile et renvoie le bon résultat sur plusieurs exemples.
- **Dichotomie** : le programme respecte le nombre d'essais maximum de chaque intervalle.
- **Jour 2** : chaque méthode est accompagnée de tests unitaires qui passent.

4. Livrables

Sur un dépôt GitHub privé partagé avec les formateurs :

Livable	Attendu	Fait ?
Dépôt GitHub	Projet Java, historique de commits lisible.	☐
Code des exercices	Les méthodes demandées, exécutable en ligne de commande via le main	☐
Tests unitaires (Jour 2)	Une classe de tests par exercice, tous au vert.	☐

5. Critères de performance

Checklist d'auto-positionnement. Objectif : pouvoir cocher chaque item honnêtement.

- **Typage** : je déclare mes variables avec le bon type et mon code compile sans erreur.
- **Logique** : les méthodes renvoient le bon résultat sur les exemples donnés.
- **Dichotomie** : mon programme trouve le nombre sous le nombre d'essais maximum.
- **Ligne de commande** : je lance chaque exercice en ligne de commande et je vois son résultat dans la console.
- **Autonomie** : je sais retrouver une syntaxe dont j'ai besoin en cherchant dans une ressource.
- **Tests (Jour 2)** : j'ai écrit des tests qui prouvent que mes méthodes fonctionnent.